

Research paper

A LiDAR-based human tracking and interaction framework for an autonomous pavement cleaning robot

Zhenyuan Yang , Aaron Ng , Yang Jiao , M. A. Viraj J. Muthugala *,
Mohan Rajesh Elara 

Singapore University of Technology and Design, 8 Somapah Road Singapore, 487372, Singapore

ARTICLE INFO

Keywords:

Human-following
Human-robot interaction (HRI)
LiDAR-based tracking
Posture recognition

ABSTRACT

Autonomous robots are becoming more and more important in human's daily life. Human-Robot Interaction (HRI) addresses the challenges of limited robot autonomy in dynamic settings. In human-robot interaction, human-following behavior is a key capability for robots supporting applications such as service robotics, healthcare assistance, and cleaning tasks. Most of the human-following and interaction frameworks are based on pure vision methods, which have limitations such as the range of sensing and different environmental conditions. This paper proposes a novel human-following framework for autonomous cleaning robots using a 3D Light Detection and Ranging (LiDAR) sensor. The proposed framework enables real-time human tracking and following based on point cloud data, complemented by human posture recognition using depth images. Unlike traditional vision-based approaches, this system overcomes challenges in limited perception range and robustness. By recognizing human postures, the robot can adapt its actions to improve task efficiency and responsiveness. Experimental results demonstrate the framework's enhanced tracking range, robustness in diverse scenarios, and faster posture recognition processing. This study contributes to the advancement of intelligent cleaning robots with improved coordination, flexibility, and interaction capabilities.

1. Introduction

Autonomous robot plays an important role in human-centered environments. As providing full autonomy to a robot is still a difficult task as the robot cannot deal with unforeseen or unpredictable events, the interaction between humans and robots is often needed [1]. As robots increasingly integrate into everyday life, from personal assistants [2] to industrial systems [3], their ability to perceive and respond to human behavior becomes significant in Human-Robot Interaction (HRI). Among these interactions, human-following behavior is a fundamental capability that supports applications such as service robots [4], social healthcare assistance [5] and autonomous vehicles [6].

In the context of autonomous cleaning robots, HRI is particularly crucial because it addresses some limitations of robots such as limited flexibility and perception range [7]. For human-following, cleaning robots in commercial or residential settings often need it to follow human supervisors or users to operate efficiently, particularly in environments where tasks are dynamically assigned or adjusted. For instance, a cleaning robot in a shopping mall can follow a janitor to specific areas that need to be cleaned, otherwise the robot needs to map

the area and the janitor needs to assign waypoints for the robot to get there, which may cost a lot of time. Similarly, robots that follow household members in their houses can improve cleaning efficiency by focusing on specific areas or responding to user-centered cleaning tasks. By integrating human-following capabilities, cleaning robots can achieve better coordination, flexibility, and responsiveness, reducing the need for manual intervention and enabling more intelligent service delivery.

Existing human-following systems rely heavily on pure vision-based methods [8], [9]. While vision-based methods are effective in controlled environments, they face significant limitations in real-world scenarios. Challenges such as varying lighting conditions and occlusions may decrease the robustness and reliability of these systems. Additionally, as the movements of persons are unpredictable, vision-only approaches often struggle in their small Field Of View (FOV) because they may easily lose tracking. These limitations increase the need for a greater variety of solutions that integrate more kinds of sensors for human-following systems.

This paper proposes a novel human-following framework for an outdoor cleaning robot using a 3D Light Detection and Ranging (LiDAR)

* Corresponding author.

E-mail address: viraj.jagathpriya@sutd.edu.sg (M.A.V.J. Muthugala).

sensor¹. This framework can do human tracking and following based on the pointcloud captured by LiDAR, and have interaction based on human posture recognition using depth image. This framework has a wider range of tracking compared to pure vision-based methods and a higher speed of processing for posture recognition. The main contributions are as follows:

- A LiDAR-based human-following and interaction framework for pavement cleaning robots, demonstrating the effectiveness of using a single LiDAR sensor instead of vision-based approaches in outdoor environments.
- A module that converts sparse LiDAR point clouds into dense depth images, which improves the reliability of depth image-based posture recognition.
- A model trained for depth image-based posture recognition using a transfer learning approach.
- A posture-based action planner that links recognized human postures with specific robot behaviors, thus enabling interaction at the task level.

The structure of this paper is as follows: Section 2 reviews the related work. Section 3 introduces the proposed LiDAR-based human posture detection framework. Section 4 presents the path planning and control algorithms for the human-following task. Section 5 shows the experiment results and Section 6 is the conclusion and future work.

2. Related work

2.1. Approaches for human following

Research on human-following robots has explored a wide range of sensing approaches. Early works were mainly vision-based. For example, Nagumo et al. [10] demonstrated one of the earliest systems by tracking a person with LED lights using a mounted camera. Hirai et al. [11] followed a human by detecting back and shoulder features with template matching, while Yoshimi et al. [12] proposed a vision-based algorithm to identify individuals by the color and texture of clothing. Despite these methods based on vision detection, vision-based methods are also widely used in human tracking algorithms [13], [14] because of the affordability and easy integration of cameras. The latest trend is toward sensor fusion, as illustrated by Lyu et al. [15], who combined LiDAR, cameras, and Ultra-Wideband signals to improve robustness in unknown environments.

Beyond single-camera systems, multi-sensor solutions were introduced to increase robustness. Morioka et al. [16] employed a sensor network of multiple cameras, while later works integrated laser range finders and Kinect sensors for more reliable detection under cluttered environments [17,18]. Arras et al. [19] focused on laser-based leg detection for tracking and prediction. Han et al. [20] explored sound signals as an alternative way for human tracking.

From the navigation and control perspective, recent studies have also emphasized integrated stacks that include global planning, local reactive collision avoidance, and probabilistic localization for mobile robots in dynamic environments. Ayalew et al. [21] propose a hybrid BRRT*-DWA planner together with AMCL, illustrating the benefit of tightly combining planning and localization when operating under uncertainty. In addition, robust trajectory tracking control for wheeled mobile robots has been actively studied, including neural-network-enhanced nonlinear PID designs combined with backstepping control [22]. Recent studies related to control further emphasize robust motion execution under uncertainty, including an optimized backstepping fuzzy sliding-mode controller for trajectory tracking of mobile manipulators [23] and a survey of modeling and control techniques for UAVs [24]. They highlight

practical directions to strengthen the downstream control layer when integrating following behaviors into real robots.

Although there is a wide range of approaches to human-following tasks, they face important limitations. Because of the limitations of visual sensors, human-following systems often suffer from degraded performance in outdoor or dynamic environments, where lighting changes, occlusions, or sensor noise reduce reliability. Sensor fusion methods can relieve some of these issues by combining multiple sensors such as cameras, LiDAR, but they often introduce higher system complexity and computation cost, which limits their practicality in real-world deployments.

2.2. Human posture recognition

Most early posture recognition systems relied on RGB images or sequences from a single camera view. Wandt et al. [25] developed a non-rigid 3D pose estimation method that avoids skeleton models by leveraging pretrained pose priors, while Jaeggli et al. [26] employed generative models with nonlinear dimensionality reduction and Recursive Bayesian Sampling for multi-activity body pose estimation. While these methods provided a basis for 3D reconstruction, they struggled with nonlinear motions, occlusions, and noisy backgrounds.

Deep learning has recently been applied to overcome these limitations. Tekin et al. [27] used spatio-temporal CNNs combined with Kernel Ridge Regression to estimate accurate 3D poses from complex video data. Similarly, Andriluka et al. [28] proposed a hybrid Bayesian method, combining 2D joint detection, tracking-by-detection, and hierarchical Gaussian Process models to recover reliable 3D poses under occlusion. The most recent approaches involve machine learning and hybrid deep learning frameworks. Gaikwad et al. [29] built a human activity recognition system using numerical datasets and classifiers such as SVM and Random Forest for real-time posture classification. Cao et al. [30] integrated Mediapipe with a CNN-BiGRU model to jointly capture temporal, spatial, and dynamic features. Ogundokun et al. [31] proposed OptiMobileX, which enhances recognition through deep feature concatenation, while Zhang et al. [32] introduced a skeleton-guided 3D CNN model tailored for elderly-care robots.

Similarly, in 3D posture recognition, most existing methods rely heavily on single-camera inputs and deep learning frameworks, which remain vulnerable to background clutter, occlusions, and light condition changes. To address these challenges, the proposed work develops a LiDAR-based integration framework for both human following and posture recognition. By relying primarily on LiDAR sensing and introducing a module to convert sparse pointcloud into dense depth images, weaknesses of vision-only systems can be mitigated and improve posture estimation under complex environments.

3. Human tracking and posture recognition

The overview of the human tracking and posture recognition framework is shown in Fig. 1. The following subsections introduce the details of the LiDAR-based human tracking method and the human-robot posture interaction.

3.1. LiDAR-based human tracking

For the LiDAR-based human tracking, the greedy closest-point matching algorithm was applied to associate points between consecutive frames to track human movement using point cloud data. At first, the user stands at the back of the robot, presses a button, and the function will be launched. The place where the user stands will be selected as an initial Region of Interest (RoI). The point cloud inside this RoI is the tracking target. The points within the RoI will be downsampled using a voxel grid filter to reduce computational complexity while maintaining sufficient spatial resolution and tracking accuracy. The initial pointcloud is shown in Fig. 2 (a) and the downsampled pointcloud is shown in

¹ The dataset and code are available in: https://github.com/isBaymax/depth_image_posture_recognition

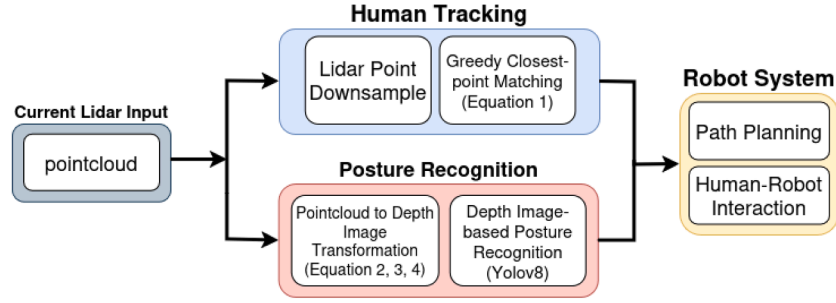


Fig. 1. The overview of the human perception framework.

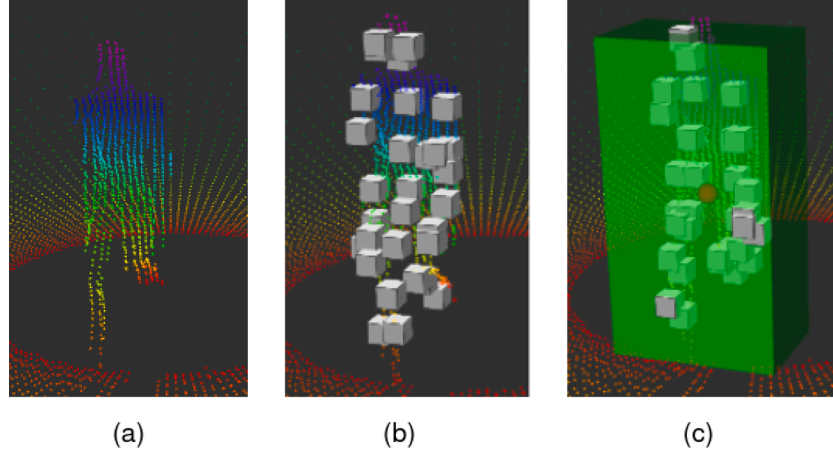


Fig. 2. An example of human tracking using pointcloud. (a) shows the original pointcloud. (b) shows the downsampled tracked points (white squares). (c) show the ROI (green box) and the centroid of the tracked points (red dot). (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

Fig. 2 (b). The ROI will move with the center of tracked points. For each subsequent frame, the set of tracked points from the previous frame is matched to the current frame's point cloud inside the ROI by calculating the Euclidean distance between every point. The Euclidean distance calculation between two points is shown in Eq. (1). x , y and z are the 3D coordinates of the points. The closest point in the current frame to each point in the last frame will be selected, provided the distance is below a predefined threshold ensuring robust association despite minor noise or movement. The whole process is shown in Algorithm 1.

$$D(p_i, p_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2 + (z_i - z_j)^2} \quad (1)$$

Algorithm 1 Human tracking.

Input: matched points in the last frame P_l ,
points in ROI P_r

Output: current tracked points P_c

```

1:  $P_d = \text{Downsample}(P_r)$ 
2:  $D_{\min} = \text{EuclideanDistance}(P_l[0], P_d[0])$ 
3: for  $p_i$  in  $P_l$  do
4:   for  $p_j$  in  $P_d$  do
5:      $D = \text{EuclideanDistance}(p_i, p_j)$ 
6:     if  $D < D_{\min}$  then
7:        $D_{\min} = D$ 
8:        $P_c.append(p_j)$ ,  $P_d.remove(p_j)$ 
9:     end if
10:  end for
11: end for
12: return  $P_c$ 

```

This iterative matching process allowed for efficient tracking of the human target over time while accounting for point displacement caused by motion. Unmatched points are considered noise or newly introduced points, and their contribution to the tracking process is disregarded. Due to its simplicity, computational efficiency, and tracking accuracy, the proposed method can demonstrate real-time capability and robust performance, even in cluttered scenes.

3.2. Pointcloud to depth image projection

After human tracking, the next step is to recognize human posture. Considering processing efficiency and real-time applications, using a depth image rather than a point cloud is more efficient, so the system needs to transfer the point cloud to a depth image format.

Firstly, a camera position is assumed. To clearly express the tracked human's position, the camera position is 5m away the center of the pointcloud of the tracked human along the x axis. In this position, the human body can be fully shown in the depth image with a proper proportion to the whole image with the proper camera intrinsic. The projection to depth image plane process uses the pinhole camera model. Suppose a pointcloud $\mathbf{P} = \{(x_i, y_i, z_i)\}_{i=1}^N$, where N is the number of points, and x_i, y_i, z_i are the coordinates of the i th point in meters under the world frame. The projection is shown in Eq. (2), where u_i, v_i are the pixel coordinates of the i th point, f_x, f_y are the focal lengths of the camera, x_c, y_c are the camera x and y coordinates, c_x, c_y are the principal point offsets of the camera. An example of the pointcloud and transferred depth image is shown in Fig. 3 (a) and (b). To express human information in this image more clearly, pointcloud outside the ROI will be filtered out to decrease the noise. The depth image after the filtering is shown in Fig. 3 (d).

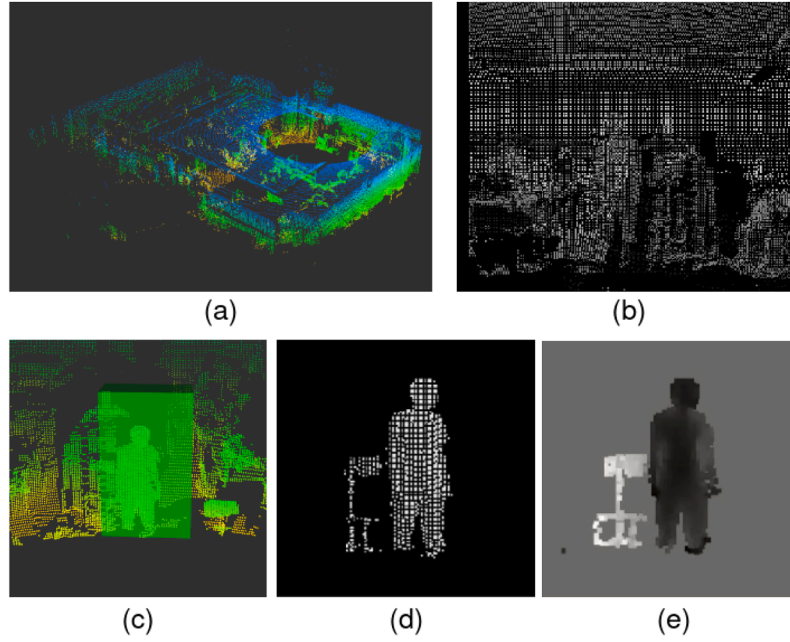


Fig. 3. The process of pointcloud to depth image projection. (a) shows the original full pointcloud. (b) shows the raw depth image. (c) shows the pointcloud inside the ROI (green box). (d) shows the depth image of the pointcloud inside the ROI. (e) shows the depth image after dilation and background overlay. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

$$u_i = \frac{f_x \cdot (y_i - y_c)}{x_i} + c_x, \quad v_i = \frac{f_y \cdot (z_i - z_c)}{x_i} + c_y \quad (2)$$

After direct pinhole projection, the image is a sparse depth image, as shown in Fig. 3 (b). To stabilize the posture recognition process, several steps need to be taken. Firstly, a dilation operation is applied using a 3×3 kernel. For each pixel value in the image D , if the pixel belongs to the detected human part, the surrounding 3×3 region of this pixel will change to this pixel value. This is to ensure the continuity in regions with valid depth data. The dilation process is shown in Eq. (3), where K is the dilation kernel. For regions lacking depth information, an adjustable background depth value d_b is overlaid. Given the image D , for each pixel (u, v) in this image, if the pixel value is bigger than 0 (means the pixel belongs to the human part), keep the value; otherwise change the pixel value to the background value d_b . The overlay process is shown in Eq. (4). The result after dilation and background overlay of the depth image is shown in Fig. 3 (e). Experimental results show that setting d_b to 15,000 maintains reliable posture recognition accuracy during the following process, as shown in Section 5. This parameter is adjustable and can be tuned according to different experimental environments.

$$D(u, v) = \max_{(i,j) \in K} I(u+i, v+j) \quad (3)$$

$$I_{\text{final}}(u, v) = \begin{cases} D(u, v) & \text{if } D(u, v) > 0 \\ d_b & \text{otherwise} \end{cases} \quad (4)$$

3.3. Depth image-based human posture recognition

In this framework, the YOLOv8-Pose [33] model is used for the customized depth image-based human posture recognition model training process. A transfer learning approach is used to perform a depth image-based posture recognition task. The YOLOv8-Pose model originally trained on the COCO dataset is a lightweight variant of the YOLOv8 architecture with high inference speed and extensibility, which is suitable for real-time and efficient processing in the human-following function. The architecture of YOLOv8-Pose is shown in Fig. 4. The backbone

Table 1

Corresponding relationship.

Posture	Action	Recognition Method
raising both arms	start follow	$v_1 - v_5 > c_1, v_1 - v_8 > c_1$
crossing 2 arms	stop follow	$v_1 - v_5 > c_1, v_1 - v_8 > c_1, u_5 - u_8 > c_2$
squatting	stop all	$v_{12} - v_1 > c_3$ or $v_{15} - v_9 > c_3$

of this architecture uses the Cross Stage Partial with two convolutions (C2F) module to improve feature propagation and reduce computational complexity. In addition, the Spatial Pyramid Pooling - Fast (SPPF) layer is used to capture multi-scale contextual information efficiently [34]. The neck is a bridge between the backbone and the head. It uses the Path Aggregation Network (PAN) to help the architecture identify features at different scales. The head is for pose estimation tasks. It predicts bounding boxes, object classes, and keypoints simultaneously. For pose estimation, the head outputs a set of keypoints for each detected object, and each keypoint has its own x and y coordinates along with a confidence score.

The system can identify different human postures by identifying the relative position of each part of the human body. There are four postures: standing, raising 2 arms up, crossing 2 arms and squatting. The postures are shown in Fig. 5. The system detects the standing position by default, and the other postures can be detected by using the corresponding relationship among human joints. The method to recognize specific posture is shown in Table 1. In Fig. 5, u and v represent the pixel coordinates in the image. The way to recognize different postures is to compare the coordinates of each joint, as shown in Table 1. In the table, u and v are the pixels, c_1 , c_2 and c_3 are three thresholds to identify each detecting condition. To recognize positions, the method is to compare them with others joints, such as comparing the head joint and the arm joints to recognize whether the hands are being raised up. The squatting detection is used for accident detection, such as when a person falls down.

3.3.1. Dataset preparation

The dataset is a customized dataset containing 642 images. These images are all depth images containing four postures: standing, raising

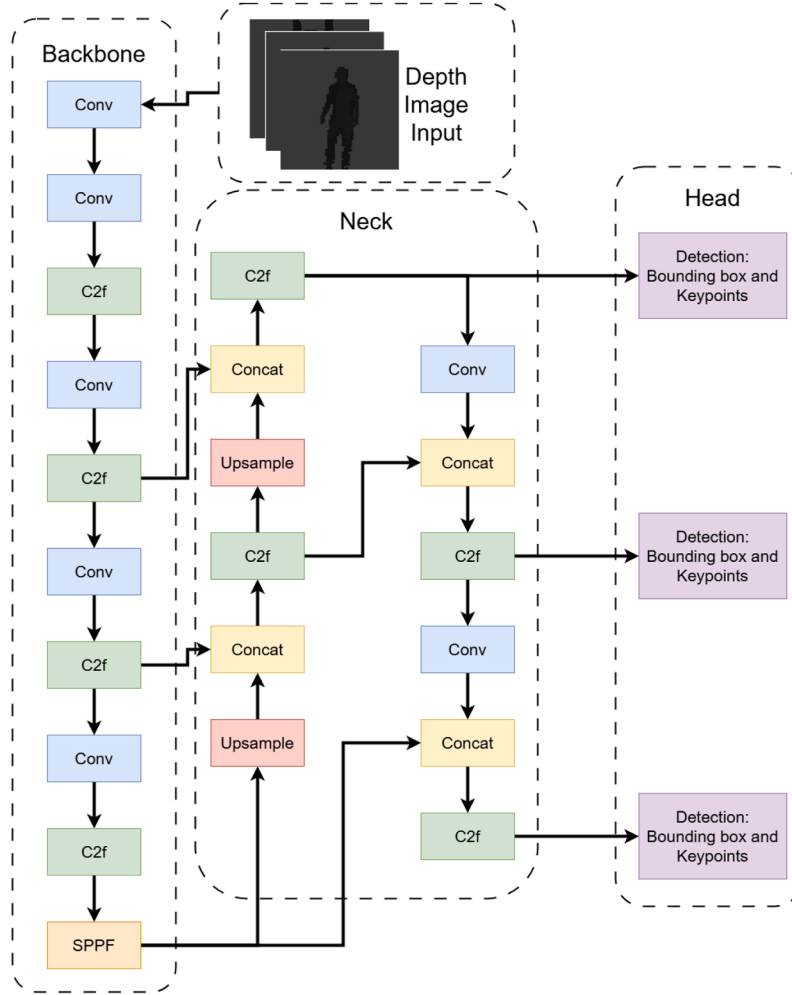


Fig. 4. The architecture of Yolov8-Pose model.

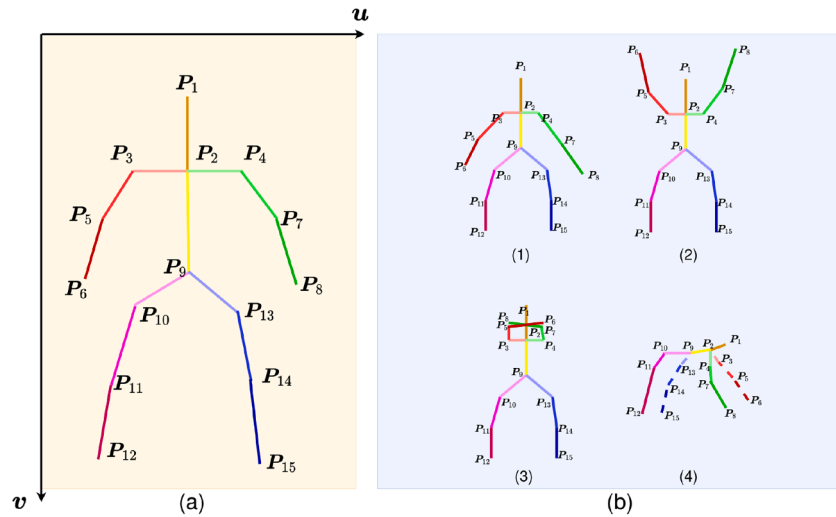


Fig. 5. The postures which are recognized in this system. (a) shows the joints of a human, 1 to 9 representing the head, neck, right shoulder, left shoulder, right elbow, left elbow, right hand, left hand, torso, right hip, left hip, right knee, left knee, right foot and left foot individually. u and v represent the pixel coordinates of the image. (b) shows all postures that need to be detected, from (1) to (4) are normal posture, raising both arms, crossing 2 arms and the falling gesture.

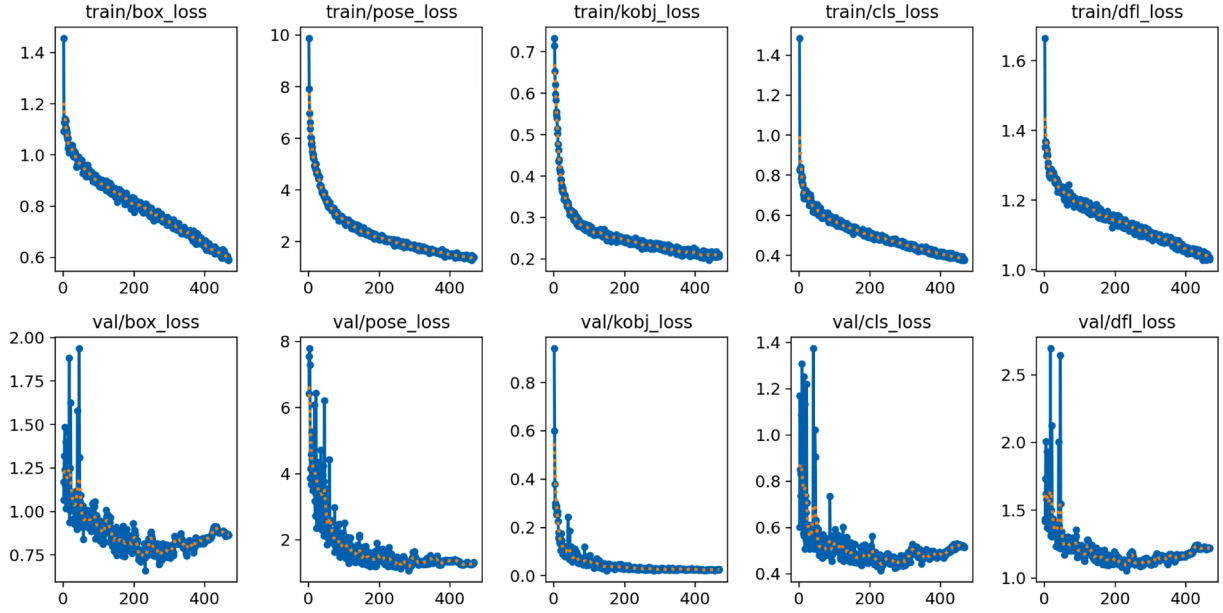


Fig. 6. The loss in the training process.

2 arms up, crossing 2 arms and squatting down. For each posture, there are 15 parts of the human body that can be recognized: head, neck, right shoulder, left shoulder, right elbow, left elbow, right arm, left arm, torso, right hip, left hip, right knee, left knee, right leg and left leg.

Given the limited scale of the dataset, potential biases may arise from subject diversity, clothing, and the viewpoint distribution induced by the LiDAR mounting configuration. To reduce overfitting and improve robustness, online data augmentation is applied during training along three dimensions: additive noise bounded by $\pm 1\%$, brightness perturbation within $\pm 25\%$, and a 1-pixel blur operator. These augmentations are designed to improve tolerance to minor projection artifacts, normalization variation, and local spatial ambiguity in the resulting depth images. After data augmentation, the size of the dataset increased up to 1542 images. The dataset is divided into the training set, validation set and testing set in a ratio of 7:2:1.

3.3.2. Model training

The model is trained on a workstation equipped with an NVIDIA GeForce RTX 3080 GPU. The model was trained for 280 epochs with a batch size of 16 and an input resolution of 640 x 640 pixels. The training used AdamW optimizer with a learning rate of 0.002 and a momentum of 0.9. The loss images are shown in Fig. 6.

There are five loss functions in the training process: box loss, pose loss, keypoint objectness loss, classification loss and distribution focal loss. Box loss is to evaluate the alignment between predicted and ground truth bounding boxes, given prediction B_p and ground truth B_g , the loss is shown in Eq. (5). ρ is the Euclidean distance between box centers, c is the diagonal of the smallest enclosing box, and v is an aspect ratio penalty with trade-off weight α . This loss encourages both high IoU and accurate box aspect ratios. The pose loss is a Mean Squared Error (MSE) over keypoints as shown in Eq. (6), where $v_k \in \{0, 1\}$ indicates visibility of keypoint k , and (x_k, y_k, c_k) and $(\hat{x}_k, \hat{y}_k, \hat{c}_k)$ are the ground truth and predicted coordinates and confidence scores, respectively. The keypoint objectness loss shows whether a person with keypoints exists in a grid cell. It uses binary cross-entropy between predicted objectness score $\hat{o}$ and the ground truth label o as shown in Eq. (7). The classification loss is the binary cross-entropy between the class label y and the prediction $\hat{y}$, which is shown in Eq. (8). The distribution focal loss is shown in Eq. (9), where $\hat{p}_i$ and $\hat{p}_{i+1}$ are the predicted probabilities for adjacent

bins around the ground truth location, and $d \in [0, 1]$ is the fractional offset.

$$\mathcal{L}_{\text{box}} = 1 - \text{IoU}(B_p, B_g) + \frac{\rho^2(\mathbf{c}_p, \mathbf{c}_g)}{c^2} + \alpha v, \quad (5)$$

$$\mathcal{L}_{\text{pose}} = \frac{1}{K} \sum_{k=1}^K v_k [(x_k - \hat{x}_k)^2 + (y_k - \hat{y}_k)^2 + (c_k - \hat{c}_k)^2], \quad (6)$$

$$\mathcal{L}_{\text{kobj}} = -[o \log(\hat{o}) + (1 - o) \log(1 - \hat{o})], \quad (7)$$

$$\mathcal{L}_{\text{cls}} = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})], \quad (8)$$

$$\mathcal{L}_{\text{dfl}} = \sum_{i=0}^{n-1} [(1 - d) \log(\hat{p}_i) + d \log(\hat{p}_{i+1})], \quad (9)$$

3.3.3. Offline test

To validate the quality of the trained depth image-based posture recognition model, the model is tested in the test dataset containing 63 images. The mean Average Precision (mAP), precision and recall are used to measure the human and keypoints detection accuracy for different confidence thresholds. The testing results are shown in Table 2. The model achieved a bounding box mAP@0.5 of 0.99, with precision and recall scores of 0.965 and 0.999, respectively. As the training process of this model only includes one “human” class, so the accuracy of the human detection is very high. For keypoint estimation, it obtained a pose mAP@0.5 of 0.972 and a pose mAP@0.5:0.95 of 0.82. The overall keypoint precision and recall were 0.943 and 0.985, respectively. These results show the capability of this model in human detection. The keypoint precision and recall reached 0.943 and 0.985 respectively, showing the consistent joint detection across varied human postures. Part of the posture recognition results are shown in Fig. 7.

4. Robot movement

After detecting the person's position, the robot should plan a path to follow the person. The path planning and robot control algorithms are introduced in this section.

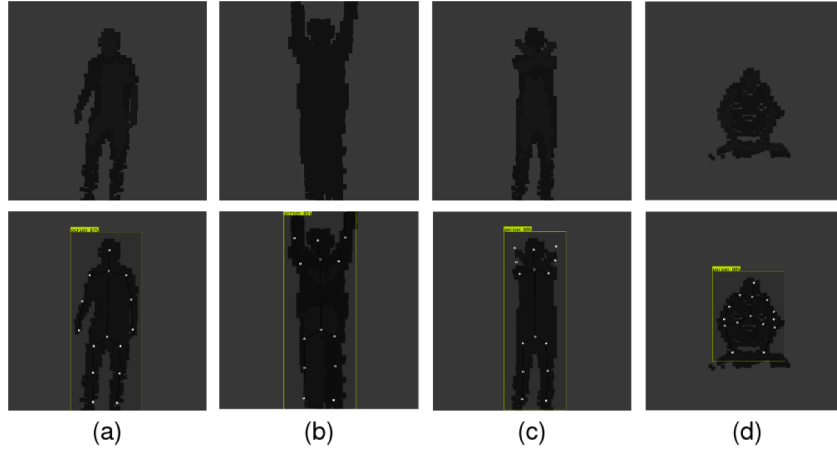


Fig. 7. The posture recognition results during the offline test. (a) is standing. (b) is raising 2 arms. (c) is crossing 2 arms. (d) is squatting.

Table 2

Validation performance of the model on the test set.

	mAP@0.5	mAP@0.5:0.95	Precision	Recall
Bounding Box	0.990	0.863	0.965	0.999
Pose	0.972	0.820	0.943	0.985

4.1. Path planning

Before following a person, the robot has to know which path to take to avoid any obstacles and follow the person at the same time. The global path planning algorithm used here is the A* algorithm [35]. It is a widely used pathfinding and graph traversal algorithm, known for its efficiency in finding the shortest path from a start node to a goal node, perfect for following the user. A* is used to compute the optimal path from the current location of the robot to a goal point. It evaluates multiple paths and selects the path with the lowest cost while avoiding obstacles or terrain. Therefore, the efficiency of A* and the ability to avoid obstacles ensure that it can follow humans safely and smoothly, making it a suitable algorithm for this study.

4.2. Robot control

Pure pursuit is a path-following algorithm commonly used in mobile robotics and autonomous vehicles [36]. The robot will follow the pre-defined path by computing control commands to steer towards the target point ahead. This target point is known as the lookahead point. The algorithm calculates the curvature of the path needed to move the vehicle from its current position to a lookahead point that is a set distance ahead of it.

Let the planned global path be a polyline represented by waypoints $\{p_k\}_{k=1}^K$, where $p_k = (x_k, y_k)$, x_k, y_k are the 2D coordinates under the world frame. Given the robot pose (x_r, y_r, ψ_r) and a lookahead distance L_d , the lookahead point p_L is selected as the first point along the path whose accumulated arc distance from the closest point on the path to (x_r, y_r) reaches L_d . The lookahead point in the robot frame (x_L, y_L) is calculated using the rigid transform, which is shown in (10). The curvature κ of the circular arc that connects the robot to the lookahead point is shown in (11). Given the commanded linear velocity v , the angular velocity command ω is set as $v\kappa$, which makes tracking smooth while updating p_L online as the robot progresses.

$$\begin{bmatrix} x_L \\ y_L \end{bmatrix} = \begin{bmatrix} \cos \psi_r & \sin \psi_r \\ -\sin \psi_r & \cos \psi_r \end{bmatrix} \begin{bmatrix} x_{p_L} - x_r \\ y_{p_L} - y_r \end{bmatrix}. \quad (10)$$

$$\kappa = \frac{2y_L}{L_d^2}. \quad (11)$$

The algorithm generates steering control based on the curvature of the path and gradually adjust the robot's headings to follow the path smoothly. As the robot moves, the lookahead point shifts forward, and the algorithm continuously updates the steering commands. In summary, pure pursuit capability in ensuring smooth trajectory and dynamic adaptability is a suitable algorithm for human-following.

The overall workflow of the human following and interaction pipeline is shown in Algorithm 2.

Algorithm 2 Overall human following and interaction.

Input: $LiDAR$ $scan$ P_t , $robot$ $pose$ $s_t = (x_r, y_r, \psi_r)$, $costmap$ $\mathcal{M}$, $lookahead$ distance L_d , $linear$ speed v
Output: $robot$ command (v_t, ω_t) , $action$ state a_t

- 1: $P_r = ExtractROI(P_t)$
- 2: **if** $t == 0$ **then**
- 3: $P_{t-1}^{trk} = P_r$
- 4: **end if**
- 5: $P_t^{trk} = HumanTracking(P_{t-1}^{trk}, P_r)$ (Eq. 1, Algorithm 1)
- 6: $c_t = \frac{1}{|P_t^{trk}|} \sum_{p \in P_t^{trk}} p$
- 7: $D_t = ProjectToDepth(P_t^{trk})$ (Eq. 2)
- 8: $D_t^{dil} = Dilate(D_t)$ (Eq. (3))
- 9: $D_t^{final} = OverlayBackground(D_t^{dil})$ (Eq. 4)
- 10: $K_t = YOLOPose(D_t^{final})$
- 11: $posture_t = RuleBasedPosture(K_t)$ (Table 1)
- 12: $a_t = MapPostureToAction(posture_t)$
- 13: **if** $a_t == follow$ **then**
- 14: $g_t = SetGoalNearHuman(c_t)$
- 15: $path_t = AStar(s_t, g_t, \mathcal{M})$
- 16: $p_L = SelectLookaheadPoint(path_t, s_t, L_d)$
- 17: $\Delta x = x_{p_L} - x_r, \Delta y = y_{p_L} - y_r$
- 18: $x_L = \cos \psi_r \cdot \Delta x + \sin \psi_r \cdot \Delta y$ (Eq. (10))
- 19: $y_L = -\sin \psi_r \cdot \Delta x + \cos \psi_r \cdot \Delta y$ (Eq. (10))
- 20: $\kappa = \frac{2y_L}{L_d^2}$ (Eq. (11))
- 21: $\omega_t = v\kappa, v_t = v$
- 22: **else**
- 23: $v_t = 0, \omega_t = 0$
- 24: **end if**
- 25: **return** (v_t, ω_t, a_t)



Fig. 8. The Panthera robot.

5. Experiments and results

5.1. Panthera platform

The experiments were conducted using the robot Panthera. Panthera is an outdoor pavement sweeping robot. It is designed for cleaning outdoor pavement. The overall size of the robot is $125 \times 204 \times 172$ cm. An industrial computer, Neusys NUVO-1018GC, used to steer the entire operation of the robot, including autonomous navigation and controlling all other sensors. In addition, the Ouster OS1-32-U LiDAR scanner is used for localization and mapping of the environment and human tracking. The robot locomotion consists of a differential wheel drive mechanism with one supporting caster wheel. Panthera has a computer with an Intel Core i7-8700 CPU @ 3.20GHz $\times$ 12 processors and an NVIDIA GeForce RTX 3080 graphic which can be used for high-level task such as computer vision. The operating system is Ubuntu 20.04. The Fig. 8 shows the Panthera robot.

5.2. Robustness analysis

In the robustness analysis, the distance between the person and the robot is considered because it is an important factor in the human-following task. In this analysis, the proposed method is compared with two vision-based posture recognition methods: MediaPipe [37] and AlphaPose [38]. In this experiment, a human walks from 3 m to 15 m away from the robot, and uses these three methods to show what are the confidence level of each keypoint in the posture recognition. The results are shown in Fig. 9 and Table 3.

The results show that the proposed model have the best performance in terms of detection robustness and accuracy compared with MediaPipe and AlphaPose. The proposed method has the highest detection confidence level, and the lowest standard deviation, which shows the lowest uncertainty. MediaPipe has a low confidence level when the human dis-

Table 3

Mean and standard deviation of the experiment results.

	MediaPipe	AlphaPose	the proposed method
Mean	0.916	0.813	0.971
Standard Deviation	0.071	0.022	0.015

Table 4

Tracking robustness comparison with LiDAR-based baselines.

Method	N_{frag} (↓)	$\bar{L}_{\text{gap}}$ (frames)	$\text{P95}(\ \hat{a}_t\ _2)$ (m/s^2)	$\text{P95}(\sigma_t)$ (m)
Euclidean	6	12.4	4.8	0.85
DBSCAN	4	8.1	3.6	0.72
Proposed	2	3.2	1.9	0.55

tance is too close, and totally loses tracking when the person is about 10 m away. AlphaPose has lower detection confidence level than the proposed method, and its confidence level has an obvious decrease when the distance increases.

5.3. LiDAR tracking robustness analysis

To provide a robustness evaluation for the LiDAR tracking, the proposed greedy association tracker is compared with two representative LiDAR-only baselines under the same test sequences and ROI initialization. The first baseline applies Euclidean clustering on ROI points and performs nearest-centroid association across frames. The second baseline applies DBSCAN clustering on ROI points and performs nearest-centroid association across frames. The performances are evaluated using track continuity and temporal stability.

Let $\hat{c}_t \in \mathbb{R}^3$ denote the estimated human centroid at time t , and let Δt be the sampling interval. A tracking gap is registered when the tracker returns no valid human cluster or point set at time t . The fragment count N_{frag} is the number of continuous time segments with valid tracking outputs, and the mean gap length $\bar{L}_{\text{gap}}$ is the average duration, measured in frames, of all gap intervals. Temporal stability is quantified using the centroid acceleration defined in (12). To emphasize tail events which related to robustness, the 95th percentile (P95) is used, defined as the value below which 95% of samples fall. This statistic is less sensitive than the maximum while still capturing occasional large deviations. Accordingly, $\text{P95}(\|\hat{a}_t\|_2)$ is used to summarize acceleration spikes, where larger values indicate sudden track jumps. Spatial consistency of the tracked point set P_t is measured by the spread defined in (13), and $\text{P95}(\sigma_t)$ is used, where larger values indicates unstable association. Table 4 summarizes the comparison.

$$\hat{a}_t = \frac{\hat{c}_t - 2\hat{c}_{t-1} + \hat{c}_{t-2}}{\Delta t^2}, \quad (12)$$

$$\sigma_t = \sqrt{\frac{1}{|P_t|} \sum_{p \in P_t} \|p - \hat{c}_t\|_2^2}, \quad (13)$$

The results show that the proposed greedy association tracker achieves the strongest robustness among the compared methods in terms of both continuity and stability. Compared with Euclidean clustering and DBSCAN baselines, the proposed method has fewer tracking interruptions ($N_{\text{frag}} = 2$ vs. 6 and 4) and shorter recovery gaps ($\bar{L}_{\text{gap}} = 3.2$ frames vs. 12.4 and 8.1), indicating more reliable target retention under sparse returns and transient disturbances. In addition, the proposed tracker shows smaller tail instability, with $\text{P95}(\|\hat{a}_t\|_2) = 1.9 \text{ m/s}^2$ (vs. 4.8 and 3.6), implying fewer sudden centroid jumps that typically arise from erroneous reassociation, and a lower tail spread $\text{P95}(\sigma_t) = 0.55 \text{ m}$ (vs. 0.85 and 0.72), suggesting reduced point-set contamination and more stable association. Overall, these results support that the proposed LiDAR tracking component is robust enough for the integrated human-following pipeline, providing stable tracking performance during continuous human motion. For the clustering-based methods, they are more

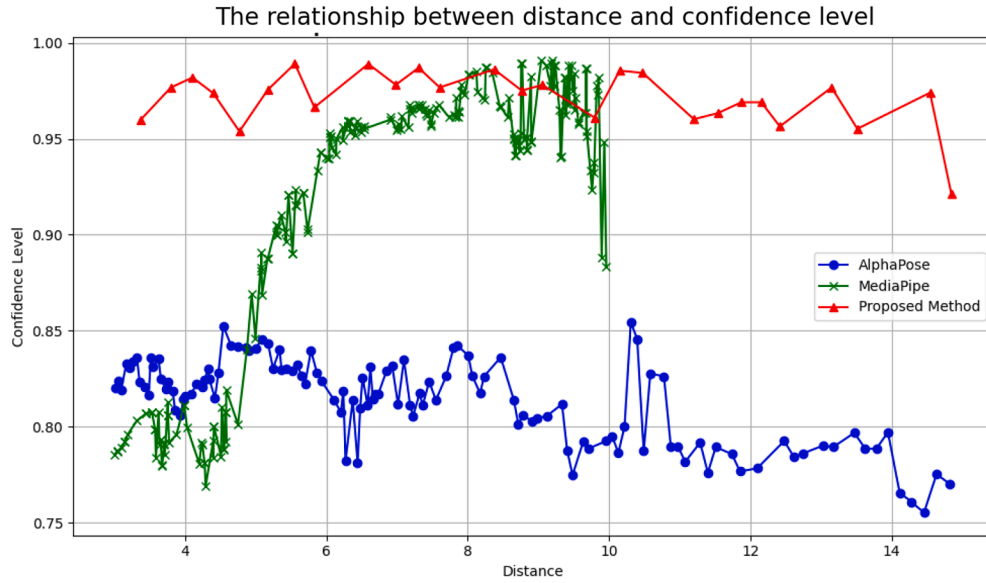


Fig. 9. The diagram shows the relationship of the human distance and the model confidence level. The red line is the proposed method, the green line is MediaPipe and the blue line is AlphaPose. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

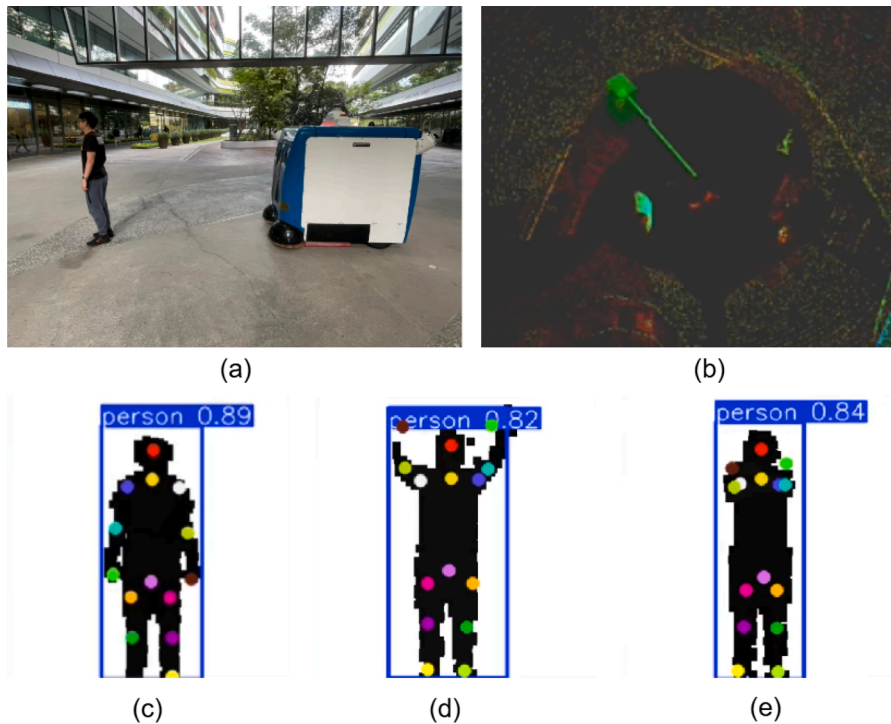


Fig. 10. The scene of the experiment. (a) is performing the human-following task using the Panthera robot. (b) is path planning (the path is the green line, the tracked person is in the green bounding box). (c), (d) and (e) are the posture recognition results of the standing position, raising arms up and crossing arms. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

sensitive to point density fluctuations and occlusions, which can cause cluster fragmentation and thus cause association failures and centroid jitter.

5.4. Time analysis

To evaluate the real-time performance of the proposed framework, the average execution time of each functional component in the human-following pipeline is considered. The system is divided into five key

stages: (1) sensor data acquisition; (2) human tracking; (3) human posture recognition; (4) path planning. In this experiment, the robot Panthera is performing an one minute human-following task. The experiment scene is shown in Fig. 10. The time of each stage during the human-following task is recorded. The average time of each stage is shown in Table 5.

The results show that the update time per LiDAR frame is 0.36 s, corresponding to an overall rate of 2.78 Hz. Importantly, the main latency comes from Stage 1 (0.254 s), which mainly reflects the LiDAR

Table 5

The time cost of each stage of the human-following pipeline.

Stage 1	Stage 2	Stage3	Stage4	Overall
0.254 s	0.056 s	0.016 s	0.034 s	0.36 s

scan period and driver-level buffering required to obtain a complete point cloud and is therefore largely constrained by the hardware. To avoid conflating sensing delay with algorithmic efficiency, the remaining stages are interpreted as processing latency, which sums to 0.106 s per frame (Stages 2–4), corresponding to a rate of only processing, which is approximately 9.43 Hz. Within the processing time, human tracking and path planning together take 0.090 s per frame, indicating that the core online decision components are lightweight and suitable for real-time deployment.

5.5. Limitations and discussion

This system has some limitations. The LiDAR-based tracking may struggle in highly cluttered or crowded environments where dynamic obstacles frequently interfere with the human target. Also the depth image-based posture recognition may not be stable if the person goes too far, as the pointcloud density will decrease a lot. In addition, using depth images cannot differentiate whether the person is facing front or back, so currently it cannot differentiate left and right side (shoulders, arms, etc).

There are three typical failure cases of this method. Tracking pointcloud becomes too sparse, the observed posture is outside the four pre-defined classes, or different body types or clothing. Specifically, when the human distance is far or under partial occlusion, the ROI may contain only a small number of pointclouds, producing fragmented projected depth images and unstable keypoint extraction. In addition, intermediate or mixed postures (e.g., partial arm raising or asymmetric configurations) are not represented in the dataset and may be forced into the closest class, potentially triggering incorrect actions. Finally, biases may remain with respect to subject diversity, limb proportions, and clothing, which can shift the projected silhouette and point distribution and reduce generalization. These limitations indicate that wider data coverage and more explicit handling of other postures are required for deployment in more diverse real-world scenarios.

In addition, there is one more limitation about the ambiguity of the front and back sides. The depth images projected from pointclouds cannot identify the human front and back sides. This ambiguity may affect actions triggered by postures. For example, when “crossing 2 arms” is observed from the back and the inferred left/right configuration becomes unstable across frames, potentially leading to incorrect or jittery action triggering.

6. Conclusion

This paper presents a LiDAR only human following and interaction framework for an outdoor pavement cleaning robot. The system tracks a target human directly in 3D point clouds through lightweight greedy association and recognizes postures by projecting the tracked points into depth images and applying a YOLOv8 Pose model trained by transfer learning. Experiments on the Panthera platform demonstrate improved robustness over representative vision based posture pipelines as the human robot distance increases, while maintaining real time feasibility with a clear breakdown of sensing and computation latency. Typical limitations and failure modes are discussed, and future work will focus on improving robustness in crowded environments and enhancing posture stability under varying viewpoints. In addition, to increase the processing speed, the overall update rate can be achieved by reducing processing latency, for example by adopting lighter-weight posture networks or applying model quantization and accelerated inference toolchains.

Finally, to solve the problem of ambiguity of identify human front and back sides, a short temporal history for posture smoothing can be introduced, or estimating coarse torso orientation directly from the tracked point cloud using simple geometric cues, thereby reducing flips without changing the sensing modality.

CRedit authorship contribution statement

Zhenyuan Yang: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Aaron Ng:** Writing – review & editing, Validation, Data curation, Conceptualization; **Yang Jiao:** Writing – review & editing, Validation, Data curation, Conceptualization; **M. A. Viraj J. Muthugala:** Writing – review & editing, Validation, Supervision, Methodology, Conceptualization; **Mohan Rajesh Elara:** Supervision, Resources, Project administration, Funding acquisition, Conceptualization.

Data availability

The dataset and code are available in: https://github.com/isBaymax/depth_image_posture_recognition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research is supported by the National Robotics Programme under its National Robotics Programme (NRP) BAU, PANTHERA 2.0: Deployable Autonomous Pavement Sweeping Robot through Public Trials, Award No. M23NBK0065, the National Robotics Programme under category National Robotics Programme 2.0, LEO 1.0: A New Class of Bed Making Robot, Award No. M25N4N2028, and also supported by A*STAR under its RIE2025 IAF-PP programme, Modular Reconfigurable Mobile Robots (MR)2, Grant No. M24N2a0039.

Supplementary material

Supplementary material associated with this article can be found in the online version at [10.1016/j.rineng.2026.109481](https://doi.org/10.1016/j.rineng.2026.109481).

References

- [1] M. Selvaggio, M. Cognetti, S. Nikolaidis, S. Ivaldi, B. Siciliano, Autonomy in physical human-robot interaction: a brief survey, *IEEE Rob. Autom. Lett.* 6 (4) (2021) 7989–7996.
- [2] A. Mohebbi, Human-robot interaction in rehabilitation and assistance: a review, *Curr. Rob. Rep.* 1 (3) (2020) 131–144.
- [3] E. Coronado, T. Kiyokawa, G.A.G. Ricardez, I.G. Ramirez-Alpizar, G. Venture, N. Yamanobe, Evaluating quality in human-robot interaction: a systematic search and classification of performance and human-centered factors, measures and metrics towards an industry 5.0, *J. Manuf. Syst.* 63 (2022) 392–410.
- [4] M.J. Islam, J. Hong, J. Sattar, Person-following by autonomous robots: a categorical overview, *Int. J. Rob. Res.* 38 (14) (2019) 1581–1618.
- [5] A. Tomoya, S. Nakayama, A. Hoshina, M. Sugaya, A mobile robot for following, watching and detecting falls for elderly care, *Procedia Comput. Sci.* 112 (2017) 1994–2003.
- [6] L. Kong, G. Jiang, Y. Tang, Y. Ou, S. Xu, Person following with dynamic obstacle avoidance, in: 2023 IEEE International Conference on Real-time Computing and Robotics (RCAR), IEEE, 2023, pp. 811–816.
- [7] T. Wang, P. Zheng, S. Li, L. Wang, Multimodal human-robot interaction for human-centric smart manufacturing: a survey, *Advanced Intelligent Systems* 6 (3) (2024) 2300359.
- [8] R. Algabri, M.-T. Choi, Deep-learning-based indoor human following of mobile robot using color feature, *Sensors* 20 (9) (2020) 2699.
- [9] I. Condés, J.-M. Cañas, E. Perdices, Embedded deep learning solution for person identification and following with a robot, in: *Advances in Physical Agents II: Proceedings of the 21st International Workshop of Physical Agents (WAF 2020)*, November 19–20, 2020, Alcalá de Henares, Madrid, Spain, Springer, 2021, pp. 291–304.

- [10] Y. Nagumo, A. Ohya, Human following behavior of an autonomous mobile robot using light-emitting device, in: Proceedings 10th IEEE International Workshop on Robot and Human Interactive Communication. ROMAN 2001 (Cat. No. 01TH8591), IEEE, 2001, pp. 225–230.
- [11] N. Hirai, H. Mizoguchi, Visual tracking of human back and shoulder for person following robot, in: Proceedings 2003 IEEE/ASME International Conference on Advanced Intelligent Mechatronics (AIM 2003), 1, IEEE, 2003, pp. 527–532.
- [12] T. Yoshimi, M. Nishiyama, T. Sonoura, H. Nakamoto, S. Tokura, H. Sato, F. Ozaki, N. Matsuhira, H. Mizoguchi, Development of a person following robot with vision based target detection, in: 2006 IEEE/RSJ International Conference on Intelligent Robots and Systems, IEEE, 2006, pp. 5286–5291.
- [13] N. Dalal, B. Triggs, Histograms of oriented gradients for human detection, in: 2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05), 1, Ieee, 2005, pp. 886–893.
- [14] A.A. Mekonnen, C. Briand, F. Lerasle, A. Herbulot, Fast HOG based person detection devoted to a mobile robot with a spherical camera, in: 2013 IEEE/RSJ International Conference on Intelligent Robots and Systems, IEEE, 2013, pp. 631–637.
- [15] H. Lyu, W. Wu, A robust human-following system for autonomous mobile robot in unknown environments, IEEE Sens. J. 25 (2025) 22739–22746.
- [16] K. Morioka, J.-H. Lee, H. Hashimoto, Human-following mobile robot in a distributed intelligent sensor network, IEEE Trans. Ind. Electron. 51 (1) (2004) 229–237.
- [17] F. Hoshino, K. Morioka, Human following robot based on control of particle distribution with integrated range sensors, in: 2011 IEEE/SICE International Symposium on System Integration (SII), IEEE, 2011, pp. 212–217.
- [18] K. Morioka, Y. Oinaga, Y. Nakamura, Control of human-following robot based on cooperative positioning with an intelligent space, IEEE Trans. Electron. Inf. Syst. 131 (5) (2011) 1050–1058.
- [19] K.O. Arras, S. Grzonka, M. Luber, W. Burgard, Efficient people tracking in laser range data using a multi-hypothesis leg-tracker with adaptive occlusion probabilities, in: 2008 IEEE International Conference on Robotics and Automation, IEEE, 2008, pp. 1710–1715.
- [20] J. Han, T. Jin, Sound source based mobile robot control for human following in a networked intelligent space, Int. J. Contr. Autom. 8 (7) (2015) 67–74.
- [21] W. Ayalew, M. Menebo, C. Merga, L. Negash, Optimal path planning using bidirectional rapidly-exploring random tree star-dynamic window approach (BRRT*-DWA) with adaptive Monte Carlo localization (AMCL) for mobile robot, Eng. Res. Express 6 (3) (2024) 035212.
- [22] T.B. Arega, Y.M. Tesfa, C.M. Abdissa, Three-wheeled mobile robot trajectory tracking control using nonlinear PID controller based neural network combined with backstepping controller, IEEE Access 13 (2025) 100167–100182.
- [23] G. Menyechel Eneyew, W. Ayalew Asfaw, C. Merga Abdissa, Optimized backstepping fuzzy sliding mode controller for trajectory tracking of mobile manipulator, Eng. Rep. 7 (7) (2025) e70269.
- [24] E.A. Gedefaw, N.B. Abera, C.M. Abdissa, A review of modeling and control techniques for unmanned aerial vehicles, Eng. Rep. 7 (6) (2025) e70215.
- [25] B. Wandt, H. Ackermann, B. Rosenhahn, 3D reconstruction of human motion from monocular image sequences, IEEE Trans. Pattern Anal. Mach. Intell. 38 (8) (2016) 1505–1516.
- [26] T. Jaeggli, E. Koller-Meier, L. Van Gool, Learning generative models for multi-activity body pose estimation, Int. J. Comput. Vis. 83 (2009) 121–134.
- [27] B. Tekin, A. Rozantsev, V. Lepetit, P. Fua, Direct prediction of 3D body poses from motion compensated sequences, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 991–1000.
- [28] M. Andriluka, S. Roth, B. Schiele, Monocular 3D pose estimation and tracking by detection, in: 2010 IEEE Computer Society Conference on Computer Vision and Pattern Recognition, Ieee, 2010, pp. 623–630.
- [29] S. Gaikwad, S. Bhatlawande, S. Shilaskar, A. Solanke, A computer vision-approach for activity recognition and residential monitoring of elderly people, Med. Novel Technol. Devices 20 (2023) 100272. <https://doi.org/10.1016/j.medntd.2023.100272>
- [30] X. Cao, X. Wang, X. Geng, D. Wu, H. An, An approach for human posture recognition based on the fusion PSE-CNN-BiGRU model, Comput. Model. Eng. Sci. 140 (1) (2024) 385–408. <https://doi.org/10.32604/cmescs.2024.046752>
- [31] R.O. Ogundokun, R. Damasevicius, R. Maskeliunas, OptiMobileX: optimizing deep transfer learning model for accurate human posture recognition using a deep feature fusion technique, IEEE Sens. J. 25 (2024) 9759–9766.
- [32] D. Zhang, Y. Zhang, M. Zhou, Skeleton-guided action recognition with multistream 3D convolutional neural network for elderly-care robot, Adv. Intell. Syst. 5 (12) (2023) 2300326.
- [33] G. Jocher, A. Chaurasia, J. Qiu, Ultralytics YOLO, 2023. <https://github.com/ultralytics/ultralytics>
- [34] Y. Liu, Z. Fu, L. Song, H. Sui, B. Li, Improved YOLOv8-pose algorithm for albacore tuna (Thunnus alalunga) fork length extraction and weight estimation, J. Mar. Sci. Eng. 12 (5) (2024) 784.
- [35] D. Foead, A. Ghifari, M.B. Kusuma, N. Hanafiah, E. Gunawan, A systematic literature review of A* pathfinding, Procedia Comput. Sci. 179 (2021) 507–514.
- [36] R.C. Coulter, Implementation of the pure pursuit path tracking algorithm. Robotics Institute Technical Report (CMU-RI-TR-92-01) (1992).
- [37] C. Lugaresi, J. Tang, H. Nash, C. McClanahan, E. Uboweja, M. Hays, F. Zhang, C.-L. Chang, M.G. Yong, J. Lee, et al., MediaPipe: a framework for building perception pipelines, arXiv preprint arXiv:1906.08172 (2019).
- [38] H.-S. Fang, J. Li, H. Tang, C. Xu, H. Zhu, Y. Xiu, Y.-L. Li, C. Lu, Alphapose: whole-body regional multi-person pose estimation and tracking in real-time, IEEE Trans. Pattern Anal. Mach. Intell. 45 (6) (2022) 7157–7173.